

Deployment of Analytics Solutions

Module VII

Anomaly Detection, Predictive Analytics and
Streaming Analytics, integrating analytics models
cloud/edge methods

By
Dr Shola Usharani

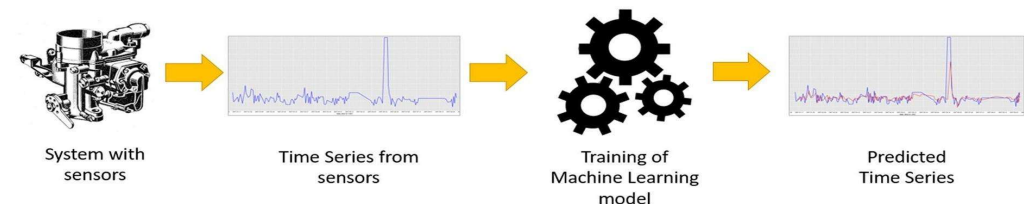
Anomaly detection

What is anomaly detection?

- An **unexpected change** within the data patterns, or an event that does not conform to the expected data pattern, is considered an anomaly.
- Anomaly detection identifies **data points, events, and/or observations that deviate from a dataset's normal behavior**.
- In other words, an anomaly is a **deviation from business as usual**.
- Anomalous data can indicate critical incidents, such as a **technical glitch**, or potential opportunities, for instance a **change in consumer behavior**.
- Machine learning is progressively being used to automate anomaly detection.
- Definition
 - Within this dataset are data patterns that represent business as usual. An unexpected change within these data patterns, or an event that does not conform to the expected data pattern, is considered an anomaly.

Why is anomaly detection?

- In IoT data, signal time series are produced by sensors strategically located on or around a mechanical component.
- A time series is the sequence of values of a variable over time.
- In this case, the variable describes a mechanical property of the object, and it is measured via one or more sensors.
- Usually, the mechanical piece is working correctly.
- As a consequence, we have **tons of examples for the piece working in normal conditions** and close to **zero examples for the piece failure**.
- This is especially true if the piece **plays a critical role in a mechanical chain because it is usually retired before any failure happens and compromises the whole machinery**.
- In IoT, a critical problem is to predict the chance of a mechanical failure before it actually happens.
- In this way, we can use the mechanical piece throughout its entire life cycle without endangering the other pieces in the mechanical chain



In enterprise IT, anomaly detection is commonly used for:

- Data cleaning
- Intrusion detection (uneven traffic patterns)
- Fraud detection (illegal transactions)
- Systems health monitoring (machine failures)
- Event detection in sensor networks (anomalies in healthcare sensor devices)
- Ecosystem disturbances
 - air quality measurements, water levels, or temperature fluctuations, to identify potential environmental hazards, pollution incidents, or natural disasters

Challenges for Anomaly detection

Anomaly detection with ML

- Machine learning, suits the engineer's purpose to create an AD system that:
 - Works better
 - Is adaptive and on time
 - Handles large datasets
- apply a machine learning algorithm to predict or recognize deterioration of mechanical pieces, or to detect cybersecurity breaches
 - The capability to predict if and when a mechanical piece will **need maintenance** leads to an optimum **maintenance** schedule and extends the lifespan of the machinery until its last breath.

1. Unstructured data: what's the anomaly?

- Structured data already implies the problem space with meaningful information.
- Anomalous data may be easy to identify because it breaks certain rules (structured data).
 - If a sensor should never read 300 degrees Fahrenheit and the data shows the sensor reading 300 degrees Fahrenheit—there's your anomaly. There is a clear threshold that has been broken.
- The data came structured, meaning people had already created an interpretable setting for collecting data.
- Applying machine learning to anomaly detection requires a good understanding of the problem, especially in situations with unstructured data.
- Unstructured data, such as **images encoded as a sequence of pixels** or **language encoded as a sequence of characters**, carry with it little interpretation and render the old algorithms useless...until the data becomes structured.

2. Large datasets needed

- principle of any good machine learning model is that it requires datasets.
- Machine learning requires datasets; **inferences can be made only when predictions can be validated.**
- Anomaly detection **benefits** from even larger amounts of data.

3.1. Supervised

- Training data is labeled with “**nominal**” or “**anomaly**”.
- The supervised setting is the **ideal setting**.
- It is the instance when a dataset comes neatly prepared for the data scientist with all data points labeled as anomaly or nominal.
- In this case, **all anomalous points are known ahead of time**. That means there are sets of data points that are anomalous, but are not identified as such for the model to train on.
- Popular ML algorithms for structured data:
 - Support vector machine learning
 - k-nearest neighbors (KNN)
 - Bayesian networks
 - Decision trees

3. Anomaly detection in three complex settings

- They all depend on the condition of the data.
- The three settings are:
 - Supervised
 - Clean
 - Unsupervised

3.2. Clean

- In the Clean setting, all data are assumed to be “**nominal**”, and it is contaminated with “anomaly” points.
- The clean setting is a **less-ideal** case where a bunch of data is presented to the modeler, and it is clean and complete, but all data are presumed to be nominal data points.
- Then, it is **up to the modeler to detect** the anomalies inside of this dataset.

3.3. Unsupervised

- In Unsupervised settings, the training data is unlabeled and consists of “**nominal**” and “**anomaly**” points.
- The **hardest** case, and the **ever-increasing case for modelers** in the **ever-increasing amounts of dark data**, is the unsupervised instance.
- The datasets in the unsupervised case do not have their parts labeled as nominal or anomalous.
- There is no ground truth from which to expect the outcome to be.
- The model must show the modeler what is anomalous and what is nominal.

- In the Unsupervised setting, **a different set of tools are needed to create order in the unstructured data.**
- In unstructured data, the **primary goal is to create clusters out of the data, then find the few groups that don't belong.**
- All anomaly detection algorithms are some form of **approximate density estimation.**
- Popular ML Algorithms for unstructured data are:
 - Self-organizing maps (SOM)
 - K-means
 - C-means
 - Expectation-maximization meta-algorithm (EM)
 - Adaptive resonance theory (ART)
 - One-class support vector machine

IoT Analytics

IoT Analytics Challenges

- One way to view IoT analytics challenges is to consider a possible IoT deployment.
 - “A huge industrial **food storage warehouse** and distribution center uses Internet-connected devices to predict and maintain the temperature of specific zones, such as a refrigeration area for items that need ongoing, non-freezing cooling, and a freezer area for items that need to be consistently frozen.”
- Too much data
 - Large volumes of data
- Security
 - Data available in public
- Misbehaving patterns
 - Abnormal behaviors
- Data Infrastructures
 - Data storage, processing it and performing analytics

1. Too much data

- The total amount of data being collected may be so large that it may not be possible to move it over the network to a central location.
 - For example, a single outside temperature sensor in the warehouse.
 - To fulfill its role it **transmits data**, including temperature, humidity, battery level, software versions, hardware versions, and motion/position changes.
 - Sensors **could transmit this information every 30 seconds, and** there could be several hundred of these sensors across the warehouse.
- This may be only one of dozens of sensor types.

3. Misbehaving devices

- These are **devices or sensors that go bad** and begin **sending false readings** to the system.
- For example, a low battery, a software bug, or a hardware failure, could cause such readings.
- This could **ruin the inventory of the warehouse.**

2. Security

- It is essential for connected devices to work together for most IoT use cases, but this approach **raises security issues.**
- The overall security profile is only as effective as the weakest device.
- If the security on a **specific vendor's outdoor sensor is weak, and the sensor is connected to other devices, the likelihood of 'indirect' critical impact is high.**
- Attackers **can compromise the sensor and modify its data or exploit the connection to other devices to cause damage.**
 - For example, a breached sensor could provide the system with an incorrect outdoor temperature reading to the system.
 - The system could adjust a zone temperature in a way that destroys the food in that area.

4 Data Infrastructure for IoT

- IoT analytics requires three key components to operate:
 - **storage,**
 - **stream processing software, and**
 - **an analytics engine.**

4.1 IoT Analytics Storage

- In an IoT architecture, there **are thousands of sensors collecting huge volumes of unstructured data**, from clickstream data to video footage.
- Modern **data streaming architectures use data lakes** like **Amazon S3** to store this raw data.
- The benefits of data lakes are that they can grow indefinitely, integrate with **many processing and analytics tools**, and provide a relatively low cost of storage.
- To enable analytics on **IoT data**, **organizations** need to plan their storage carefully.
 - Just dumping data into a data lake with no prior treatment can create a data swamp.
- **Upsolver** is a stream processing and data lake management platform that can save IoT data to a data lake in a format that enables SQL-based analysis by traditional analytics tools.

4.3 Analytics Engine

- Several vendors provide purpose-built analytics engines designed to work with IoT data.
- Two types of Analytics Engines
 - AWS IoT Analytics
 - Azure IoT Analytics

4.2 Stream Processing

- Stream processing allows to analyze continuous data flows in memory, with only required state changes or data to be transported to a database or file system.
- This process, called Change Data Capture (CDC), is useful in an IoT setting as it permits a system to **recognize relevant information while removing less useful data points**.

AWS IoT analytics

- AWS IoT analytics **transforms, filters and enriches IoT data** prior to storing it in a time-series data store for analysis.
- It collects data from devices, **transforms it into a usable form**, enriches the data with device-specific **metadata, and stores the processed data**.
- Then analyze data by initiating ad-hoc or scheduled queries using the built-in **SQL query engine, or run machine learning algorithms on the data**.
- AWS IoT analytics **includes pre-built models for common IoT use cases like predictive maintenance and smart agriculture**.

Azure IoT analytics

- Azure Stream Analytics integrates with open source cloud platforms to provide real time analytics on data from IoT applications and devices.
- Azure IoT analytics allows you to:
 - Develop massively parallel Complex Event Processing (CEP) pipelines
 - Scale instantly
 - Build real-time dashboards
 - Guarantee high availability for IoT data
 - Create compliance audits

Data Collection for IoT connected devices

- Data collection layers
- Type of data

IoT Data Collection Technology and Process

- Device Layer
- Communication Layer
- IT Edge Layer
- Event processing layer
- Client communication layer

Device Layer

- A range of devices that communicate with one another is the primary layer of IoT architecture. Examples of IoT data collection technology:
 - Sensors that track motion, temperature, heart rate, and other variables;
 - Actuators;
 - ZigBee devices;
 - Bluetooth and BLE devices;
 - Low-power-radio-based devices.
- All IoT devices have an identity that falls into one of the following categories:
 - A built-in unique identifier (or UUID) placed inside of a device, like a chip;
 - An identifier that relies on radio IoT data collection systems — Wi-Fi MAC, Bluetooth, etc.;
 - An identifier located inside the system's non-volatile memory (EEPROM);
 - A Refresh/Bearer token.

Communication Layer

- This part of the architecture allows devices to communicate with each other and exchange data. The communication layer consists of protocols, among which the following ones:
 - **HTTP/HTTPS** — a basic text-based protocol supported even by low-end 8-bit devices.
 - **MQTT** — a protocol, designed to handle embedded systems and optimized to support IoT. It is known for a wide community of followers, as well as a robust asset library.
 - **CoAP** — based on HTTP semantics, CoAP scores higher in terms of a footprint. Compared to MQTT, the protocol is harder to connect to firewalls and has poorer library support.

Event processing layer

- After IoT data is collected, an application needs to process and store it — this happens in the event processing layer of the system. In this layer, multiple operations are handled:
 - Cleansing the data;
 - Structuring gathered insights;
 - Storing the information inside a database;
 - Adding metadata to IoT data.
- There are several ways to build this part of the system
 - It design a database-powered server-side application, use a big data analytics platform on an IoT cloud service to process and store IoT data, or
 - Support real-time event processing from IoT devices.

IT Edge Layer

- This layer is often considered the application's command station, **A Gateway with OS, Data storage and device management communications.**
- The edge **helps ensure that most data processing** is happening off the connected device in a dedicated environment.
- The layer is used to support a wide network of devices that have the little-to-no processing power of their own but will supply the system with high data volumes
- In an IoT application, it carries out the following functions:
 - Device management;
 - Ensuring processing security;
 - Aggregating and replicating data;
 - Routing data to/from the cloud;
 - Priority messaging;
 - Processing images, audio, and other types of data at the edge.

Client communication layer

- In this layer of the IoT architecture, all collected data is transferred from a device-oriented to a user-oriented system.
- To relay data from IoT devices to end-users, a tech team needs to **build front-ends that interact with databases and the back-end.**
- To make sure that IoT data can interact with outside systems, developers employ machine-to-machine **APIs.**
- The most common way to relay IoT insights to an end-user is via **web or mobile applications.**

Types of IoT Sensor Data

- IoT data is the information collected by connected devices — sensors, wearables, and others.
- However, not all types of sensor data are equally complex.
- Here is the breakdown of the main insight classes a tech team can collect — from the most basic to the most advanced.
 - Status data
 - Location data
 - Automation data
 - Actionable data

Location data

- Tracking the movement of an object or a person is another important function of IoT devices and sensors.
- Connected systems use location data for fleet management, asset tracking, employee monitoring, and other management tasks.
- IoT may offer higher data processing speed and precision than GPS — that's why a lot of business owners and public office managers use motion sensors instead of GPS trackers.

Status data

- Status data is the baseline for most IoT applications.
- It's the most basic type of information gathered — whether an appliance is off or on, whether there are available spots at a property, etc.
- This data is useful for all decision-making, planning, and maintenance.
- However, it may have little value if not paired with other types of IoT data.

Automation data

- Teams will be able to allocate human resources efficiently and encourage talent to focus on carrying out demanding assignments, not routine tasks.
- This type of data helps IoT systems control devices inside a house, vehicles on the road, and other moving parts of any system.
- Having said that, once security practices and a code of conduct are established, an increasing number of IoT systems will rely on automation data.
- Processing automation data is a complex process since the stakes in case of errors are extremely high.

What Industries Benefit From Data Collection Technology

Actionable data

- These types of IoT datasets are an extension of status data.
- The system processes uses this data and transforms into easy-to-carry-out instructions.
- Actionable data is often used in forecasting and prediction, energy consumption and workplace efficiency optimization, as well as during long-term decision-making.
- Through actionable data, business owners and public officials can make better use of other insights an IoT system has captured.

- Healthcare
- Manufacturing
- Agriculture
- Energy
- Smart homes
- Transportation

Reference

- <https://www.digiteum.com/iot-data-collection/>

Types of IoT Analytics

- Descriptive analytics on IoT data
- Diagnostic analytics on IoT data
- Predictive analytics on IoT data
- Prescriptive analytics on IoT data

Top 5 Predictive Analytics Models

Predictive analytics on IoT data

- Raises the question: **what will happen?**
- Assesses the likelihood that something will happen within a specific timeframe, according to historical data.
- The aim is to **proactively take corrective action before an undesired outcome occurs, to mitigate risk, or to isolate opportunities.**
- Typically **implemented via machine learning models that are trained with historical data**, and stationed on the cloud so that they can be accessed by end-user applications.
- Addresses questions such as:
 - What's the likelihood of this machine failing in the next 24 hours?
 - What is the anticipated useful life of this tool?
 - When should I service this machine?
 - What will be the demand for this feature or product?

Classification Model

- The classification model is the **simplest of the several types** of predictive analytics models.
- It puts data in categories based on what it **learns from historical data.**
- Classification models are best to answer **yes or no questions**, providing broad analysis that's helpful for guiding decisive action.
- These models can answer questions such as:
 - For a retailer, "Is this customer about to churn?"
 - For a loan provider, "Will this loan be approved?" or "Is this applicant likely to default?"
 - For an online banking provider, "Is this a fraudulent transaction?"
- The breadth of possibilities with the classification model—and the ease by which it can be retrained with new data—means it can be applied to many different industries.

- **Classification Model**
- **Clustering Model**
- **Forecast Model**
- **Outliers Model**
- **Time Series Model**

Clustering Model

- The clustering model **sorts data into separate, nested smart groups based on similar attributes.**
 - If an ecommerce shoe company is looking to **implement targeted marketing campaigns for their customers**, they could go through the **hundreds of thousands of records to create a tailored strategy for each individual.**
 - But is this the most efficient use of time? Probably not.
- Using the clustering model, **they can quickly separate customers into similar groups based on common characteristics and devise strategies for each group at a larger scale.**
- Other use cases of this predictive modeling technique might include grouping loan applicants into **"smart buckets" based on loan attributes, identifying areas in a city with a high volume of crime**, and benchmarking SaaS customer data into groups to identify global patterns of use.

Forecast Model

- One of the most widely used predictive analytics models, the forecast model deals **in metric value prediction, estimating numeric value for new data based on learnings from historical data.**
- This model can be applied wherever historical numerical data is available. Scenarios include:
 - **A SaaS company can estimate how many customers they are likely to convert within a given week.**
 - **A call center can predict how many support calls they will receive per hour.**
 - **A shoe store can calculate how much inventory they should keep on hand in order to meet demand during a particular sales period.**
- The forecast model also considers multiple input parameters.
- If a restaurant owner wants to predict the number of customers she is likely to receive in the following week, the model will take into account factors that could impact this, such as: Is there an event close by? What is the weather forecast? Is there an illness going around?

Time Series Model

- The time series model **comprises a sequence of data points captured, using time as the input parameter.**
- It uses the last year of data to develop a numerical metric and predicts the next three to six weeks of data using that metric.
- **Use cases for this model includes the number of daily calls received in the past three months, sales for the past 20 quarters, or the number of patients who showed up at a given hospital in the past six weeks.**
- It is a potent means of understanding the way a singular metric is developing over time with a level of accuracy beyond simple averages.
- It also takes into account seasons of the year or events that could impact the metric.
- However, growth is not always static or linear, and the time series model can better model exponential growth and better align the model to a company's trend.
- It can also forecast for multiple projects or multiple regions at the same time instead of just one at a time.

Outliers Model

- The outliers model is oriented **around anomalous data entries within a dataset.**
- It can identify anomalous figures either by themselves or in conjunction with other numbers and categories.
 - Recording a spike in support calls, which could indicate a product failure that might lead to a recall
 - Finding anomalous data within transactions, or in insurance claims, to identify fraud
 - Finding unusual information in your NetOps logs and noticing the signs of impending unplanned downtime
- The outlier model is particularly **useful for predictive analytics in retail and finance.**
- For example, when identifying fraudulent transactions, the model can assess not only amount, but also location, time, purchase history and the nature of a purchase (i.e., a \$1000 purchase on electronics is not as likely to be fraudulent as a purchase of the same amount on books or common utilities).

References

- <https://insightsoftware.com/blog/top-5-predictive-analytics-models-and-algorithms/>

References

- <https://www.anodot.com/blog/what-is-anomaly-detection/>
- <https://www.bmc.com/blogs/machine-learning-anomaly-detection/>

Streaming Analytics

What is streaming or real-time analytics?

- **Streaming analytics** or **real-time analytics** is a type of data analysis that presents real-time data and allows for performing simple calculations with it.
- Working with real-time data involves different mechanisms as compared to working with historical data.
- It uses a **specific type of processing large amounts of constantly updating data**, called **stream processing**.

Source: www.altexsoft.com/blog/real-time-analytics

Batch processing (Traditional analytics processing)

- Traditional analytics mean business intelligence (BI) methods and technical infrastructure.
- BI is a practice of supporting data-driven business decision-making.
- It mainly focuses on historical data, which in most cases doesn't lose its importance or relevance.

Source: www.altexsoft.com/blog/real-time-analytics

Batch processing architecture

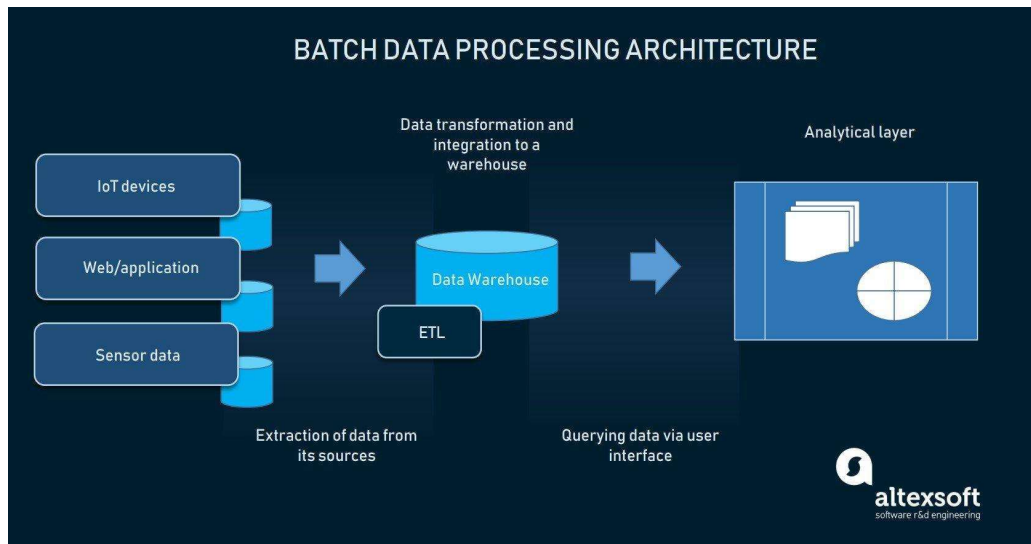
- Historical data is stored as a **stable unit that can be divided into pieces**.
- In the process of ETL and warehousing, the data is moved and processed by batches.
- A batch has to be queried by a user or a software program.
- So, the system would understand when to fetch data and which pieces of it, how to process it, and present it to the end user.

Source: www.altexsoft.com/blog/real-time-analytics

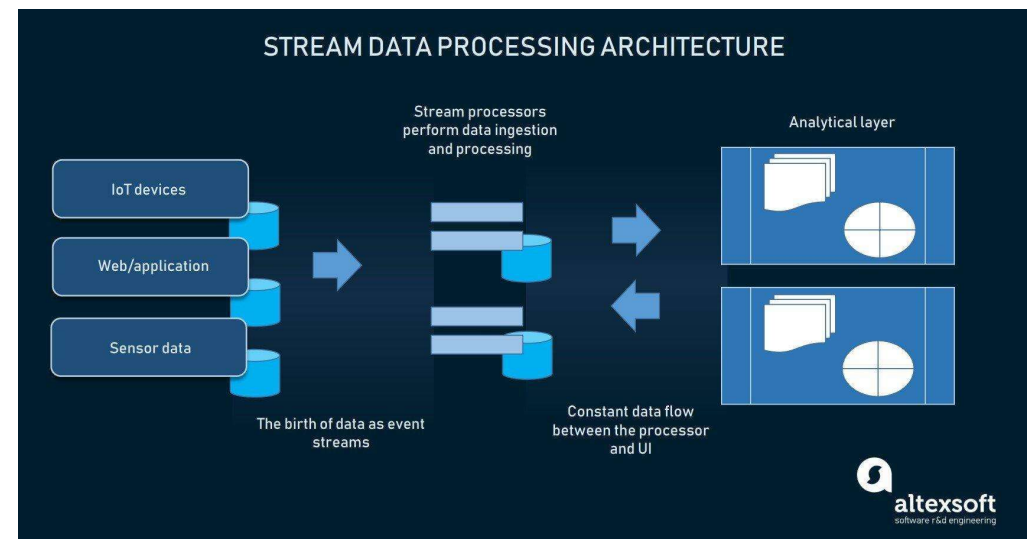
Stream processing

- Streaming processing deals with **data streams**.
- A data stream is a constant flow of data, which updates with high frequency and loses its relevance in a short period of time.
- For example, these could be transactional data, information from IoT devices, hardware sensors, etc.
- **As data streams have no beginning or end, they can't be broken into batches.**
- So there is no time when the data can be uploaded into storage and processed.
- Instead, data streams are processed on the fly.

Source: www.altexsoft.com/blog/real-time-analytics



Source: www.altexsoft.com/blog/real-time-analytics



Source: www.altexsoft.com/blog/real-time-analytics

How Does Stream Analytics Work?

- Streaming analytics, also known as **event stream processing**, is the analysis of huge pools of current and “**in-motion**” data through the use of continuous queries, called event streams.
- These streams are triggered by a specific event that happens as a direct result of an action or set of actions, like a financial transaction, equipment failure, a social post or a website click or some other measurable activity.
- The data can originate from the Internet of Things (IoT), transactions, cloud applications, web interactions, mobile devices, and machine sensors.
- By using streaming analytics platforms, organizations can extract business value from data in motion just like traditional analytics tools would allow them to do with data at rest.
- Real-time streaming analytics help a range of industries by spotting opportunities and risks.

Source: databricks.com/glossary/streaming-analytics



Source: databricks.com/glossary/streaming-analytics

The Advantages of Streaming Analytics

- **Data visualization.** Keeping an eye on the most important company information can help organizations manage their key performance indicators (KPIs) on a daily basis. Streaming data can be monitored in real time allowing companies to know what is occurring at every single moment
- **Business insights.** In case an out of the ordinary business event occurs, it will first show up in the relevant dashboard. It can be used in cybersecurity, to automate detection and response to the threat itself. This is an area where abnormal behavior should be flagged for investigation right away.
- **Increased competitiveness.** Businesses looking to gain a competitive advantage can use streaming data to discern trends and set benchmarks faster. This way they can outpace their competitors who are still using the sluggish process of batch analysis.
- **Cutting preventable losses.** With the help of streaming analytics, we can prevent or at least reduce the damage of incidents like security breaches, manufacturing issues, customer churn, stock exchange meltdowns, and social media crisis.
- **Analyzing routine business operations.** Streaming analytics offers organizations an opportunity to ingest and obtain an instant insight from the real-time data that is pouring in.

Source: databricks.com/glossary/streaming-analytics

Source: databricks.com/glossary/streaming-analytics

Example Scenario

- In a smart agriculture setting, IoT sensors are deployed throughout a farm to monitor various environmental conditions and optimize crop production. The farm owner, Jane, utilizes an IoT data stream application to collect, analyze, and visualize real-time data from these sensors to make informed decisions and maximize crop yield while minimizing resource usage.

Performance or
effectiveness of
analytical models :
Checking the
chosen model is
predicts correctly

Reference

- <https://databricks.com/glossary/streaming-analytics>
 - <https://www.altexsoft.com/blog/real-time-analytics/>
 - <https://cloud.google.com/learn/what-is-streaming-analytics>
 - <https://cloud.google.com/blog/products/data-analytics/how-streaming-data-analytics-works-for-real-time-processing>
-
- You can train your supervised machine learning models all day long, but unless you evaluate its performance, you can never know if your model is useful.

Why evaluation is necessary?

- A machine learning model can be trained extensively with many parameters and new techniques, but as long as you are skipping its evaluation, you cannot trust it.

How to read the Confusion Matrix?

- A **confusion matrix** is a correlation between the predictions of a model and the actual class labels of the data points.

Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Confusion Matrix for a Binary Classification

- Let's say you are building a model that detects whether a person has diabetes or not. After the train-test split, you got a test set of length 100, out of which **70 data points are labeled positive (1), and 30 data points are labelled negative (0)**. Now let me draw the matrix for your test prediction:

	Actually Positive	Actually Negative
Predicted Positive	64	3
Predicted Negative	6	27

- Out of 70 actual positive data points, your model predicted 64 points as positive and 6 as negative. Out of 30 actual negative points, it predicted 3 as positive and 27 as negative.
- In the notations, **True Positive, True Negative, False Positive, & False Negative**, notice that the second term (Positive or Negative) is denoting your prediction, and the first term denotes whether you predicted right or wrong.
- Based on the above matrix, we can define some very important ratios:
 - **TPR (True Positive Rate)** = (True Positive / Actual Positive)
 - **TNR (True Negative Rate)** = (True Negative/ Actual Negative)
 - **FPR (False Positive Rate)** = (False Positive / Actual Negative)
 - **FNR (False Negative Rate)** = (False Negative / Actual Positive)

Accuracy

- case of diabetes detection model, we can calculate these ratios:
 - TPR = 91.4%
 - TNR = 90%
 - FPR = 10%
 - FNR = 8.6%
- If you want model to be smart, then your model has to predict correctly.
- This means **True Positives** and **True Negatives** *should be as high as possible*, and at the same time, you need to minimize your mistakes for which your **False Positives** and **False Negatives** *should be as low as possible*.
- Also in terms of ratios, your **TPR & TNR** *should be very high* whereas **FPR & FNR** *should be very low*,
- **A smart model:** TPR ↑, TNR ↑, FPR ↓, FNR ↓
- **A dumb model:** Any other combination of TPR, TNR, FPR, FNR

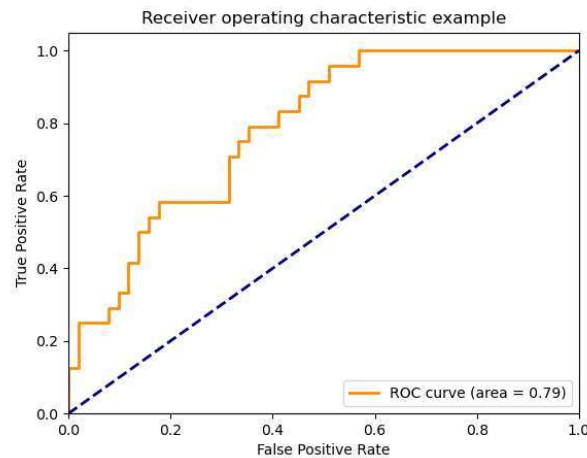
- Accuracy, a measure of how accurate your model is.
- **Accuracy = Correct Predictions / Total Predictions**
- **By using confusion matrix, Accuracy = (TP + TN)/(TP+TN+FP+FN)**
- Accuracy can sometimes lead you to false illusions about your model, and hence you should first know your data set and algorithm used then only decide whether to use accuracy or not.
- The two types of data sets:
 - **Balanced:** A data set that contains almost equal entries for all labels/classes. E.g., out of 1000 data points, 600 are positive, and 400 are negative.
 - **Imbalanced:** A data set that contains a biased distribution of entries towards a particular label/class. E.g., out of 1000 entries, 990 are positive class, 10 are negative class.
- **Very Important: Never use accuracy as a measure when dealing with imbalanced test set.**

Precision & Recall

- **Precision:** It is the ratio of True Positives (TP) and the total positive predictions. Basically, it tells us how many times your positive prediction was actually positive.
- **Recall :** It is nothing but TPR (True Positive Rate explained above). It tells us about out of all the positive points how many were predicted positive.
- **F-Measure:** Harmonic mean of precision and recall.
- To understand this, let's see this example: When you ask a query in google, it returns 40 pages, but only 30 were relevant. But your friend, who is an employee at Google, told you that there were 100 total relevant pages for that query. So it's precision is $30/40 = 3/4 = 75\%$ while it's recall is $30/100 = 30\%$. So, in this case, precision is "how useful the search results are," and recall is "how complete the results are."

ROC & AUC

- **Receiver Operating Characteristic Curve (ROC):**
- It is a plot between TPR (True Positive Rate) and FPR (False Positive Rate) calculated by taking multiple threshold values from the reverse sorted list of probability scores given by a model.



- Since the maximum TPR and FPR value is 1, the area under the curve (AUC) of ROC lies between 0 and 1.
- The area under the blue dashed line is 0.5.
- AUC = 0 means very poor model, AUC = 1 means perfect model.
- As long as model's AUC score is more than 0.5. The model is making sense because even a random model can score 0.5 AUC.
- **Very Important:** You can get very high AUC even in a case of a dumb model generated from an imbalanced data set. So always be careful while dealing with imbalanced data set.
- AUC had nothing to do with the numerical values probability scores as long as the order is maintained.
- AUC for all the models will be the same as long as all the models give the same order of data points after sorting based on probability scores.

Reference

- you should know your data set and problem very well, and then you can always create a confusion matrix and check for its accuracy, precision, recall, and plot the ROC curve and find out AUC as per your needs. But if your data set is imbalanced, never use accuracy as a measure.

- <https://analyticsindiamag.com/what-is-predictive-model-performance-evaluation-and-why-is-it-important/>
- <https://www.kdnuggets.com/2020/09/performance-machine-learning-model.html>

Implementing web based API for IoT

Three different ways

Direct integration pattern

- used for devices that support HTTP and TCP/IP and can therefore expose a web API
- useful when a device can directly connect to the internet; for example, it uses Wi-Fi or Ethernet

Gateway integration pattern

- resource-constrained devices can use non-web protocols to talk to a more powerful device (the gateway), which then exposes a REST API for those non-web
- Devices useful for devices that can't connect directly to the internet; for example, they support only Bluetooth or ZigBee
- can't serve HTTP requests directly

Cloud integration pattern

- powerful and scalable web platform to act as a gateway useful for any
- device that can connect to a cloud server over the internet, regardless of whether it uses HTTP or not,

Direct Integration pattern (REST implementation)

Uses HTTP servers

Required 50 Bytes of RAM to implement HTTP servers on minicomputer.

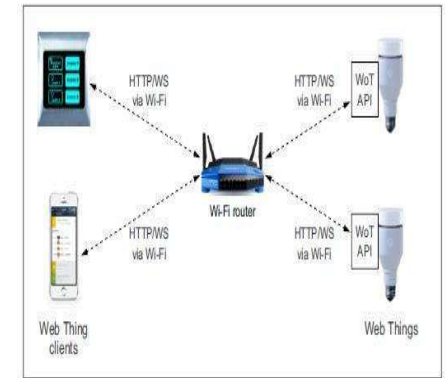
TCP/IP stack is included

Raspberry Pi can implement it through REST API.

Used :

when the device isn't battery powered and when direct access from clients such as mobile web apps is required.

Example : home automation, where power is usually available and low-latency local interactions are important—for instance, turning lights on/off.



Examples

Examples CoAP → Gateway Integration pattern

Examples MQTT → Cloud integration pattern

Deployment CoAP & MQTT

Gateway Integration pattern

Disadvantages of Direct Integration

much of a power drag

Battery powered devices, Wi-Fi or Ethernet need much of a power.

Introduction (need of Gateway Integration pattern)

For Direct Integration pattern : IoT using HTTP & WebSockets API

- Non-constrained devices
- Works fine for not battery powered,
- Having sufficient bandwidth (Wi-Fi/bandwidth)
- Having enough RAM storage.
- Support of HTTP & WS and TCP/IP

Constrained devices:

- Works fine for battery powered,
- Not having sufficient bandwidth (Wi-Fi/bandwidth)
- Not Having enough RAM storage.
- Not Support of HTTP & WS and TCP/IP

intermediate devices or Proxy devices as Gateways

- an intermediary devices that can expose the device's functionality through a web based API
 - These **intermediaries** are called application gateways or IoT gateways
- they can talk to Things using any non-web application protocols and then translate those into a clean **REST API** that any **HTTP** client can use.
- Some gateways can add a layer of **security or authentication**, aggregate and store data temporarily, expose semantic descriptions for Things
- Example :
 - a CoAP device using an HTTP and WebSockets API.

CoAP device

- CoAP is an interesting protocol based on REST
- It isn't HTTP and uses UDP instead of TCP,
- It is a gateway that translates CoAP messages from/to HTTP is needed.
- device-to-device communication over low-power radio communication
- can't talk to a CoAP device from a JavaScript application in your browser without installing a special plugin or browser extension.
- Example
 - Pi as a gateway to CoAP devices

Proxy CoAP

- Accessing the constrained device from the browser through Gateway (Pi)
- extend your IoT gateway to provide access to CoAP devices via your Pi.
- **Constrained device → Proxy CoAP (Gateway) → HTTP can access, then CoAP devices → web browsers.**

CoAP Server

simulate a CoAP device on your computer

How??

Install CoAP on a resource-constrained device such as an Arduino Uno.

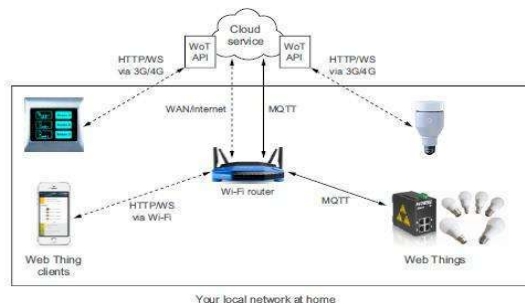
Application

mobile phone (Gateway) to upload the data from your Bluetooth bracelet, by bridging/translating various protocols.

Cloud Integration Pattern

Introduction to Cloud Integration pattern

It is an extension of the gateway pattern where the gateway is a remote server that devices and applications access via the internet



Need of Cloud Integration Pattern

- you need to manage large quantities of devices and data
- need to have a much more powerful environment
- and scalable platform to store and process the data.
- Example :
 - Industry Automation, smart city, logistic support

Advantages of cloud Integration pattern

- it **doesn't have the physical constraints of devices** and gateways,
- it's much **more scalable** and can process and store a virtually unlimited amount of data.
- A cloud platform to **support many protocols at the same time**, handle protocol translation
- support **many more concurrent clients than an IoT device platform**
- Building **complex applications in a fraction** of the time due to many features that might take considerable
- time to build from scratch, from industry-grade security, to specialized analytics capabilities, to flexible data visualization tools and user and access management.
- platforms are natively connected to the web, data and services from
- your devices can be **easily integrated into third-party systems to extend** your devices.

Supported cloud computing tools

Xively

THingWorx

THingSpeak

Carriots

thethings.io

Issues in cloud technology

- **CPUs are not sufficient to process all data** and provide accurate and real time results in time
- Optimal decisions based on **comprehensive SA (Situational Awareness) are needed**
 - Surrounding devices data and options
- Not all data is available at a node
- Collected **data size is too big to compute at one node**
- Computing at mobile devices is very less
 - Limited battery energy restricts fast & large amounts of computing

Problems in IoT data analysis

- Bring analytics closer to IoT data source in order to **remove unnecessary delays**.
- Bringing analytics closer to IoT data source puts forth a new set of **challenges, including limitation of power, storage and computing resources**.

Type of cloud technologies & edge computing technologies

- Mobile Cloud Computing (MCC)
- Edge computing (MEC)
 - Mobile Edge computing (MEC)
 - Fog Computing
 - Cloudlet

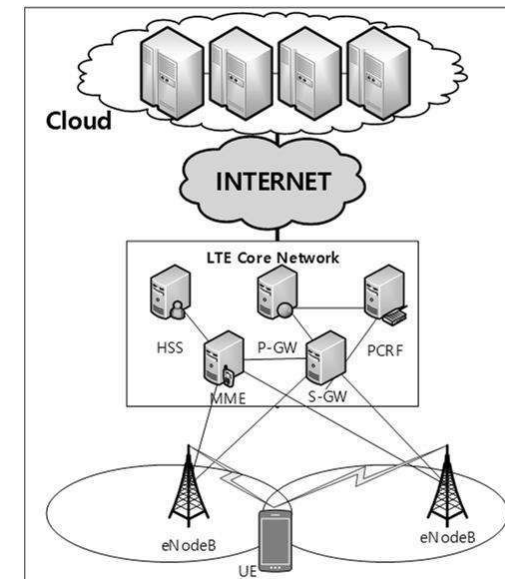
IoT & Mobile cloud technology

- Accessing huge amounts of data from different IoT devices to cloud
- Cloud computing is available at internet and is very far away from the IoT devices
- Fast , reliable and quickly responses need to bring the cloud technology near to the devices
 - Edge computing & MCC (Mobile Cloud Computing)
 - Brining the cloud resources near to the edge of the network where we are and the network to use it efficiently is edge computing

Mobile cloud computing

- MCC (Mobile Cloud Computing) allows cloud computing to mobile and IoT users
- UE (User Equipment) and IoT systems can use power computing and storage resources of a distant CC (Centralized Cloud) through a CN (Core Network) of a mobile operator and the Internet

Mobile Cloud computing communication -4G (LTE) Networks



Operation of MCC

- Distance from the UE to cloud
 - Operations from UE to base band nodes then to LTE networks finally to cloud by internet
 - **Distance of data to travel is very long**

Advantages of MCC

- Extends battery lifetime by Offloading energy consuming computations to the cloud
- Enables faster sophisticated application support (i.e., IaaS, PaaS, SaaS) to mobile users and IoT systems
 - Compute results and send because of service models
- Provides massive data storage to mobile users and IoT systems

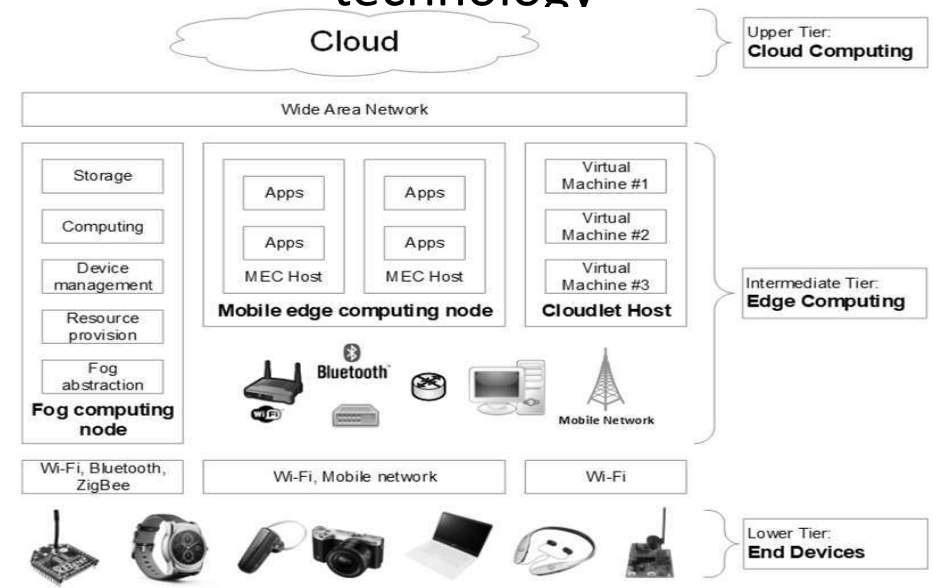
Edge computing

- Computing & storage resources are moved closer to the edge (i.e., near the BS (Base Station) or AP (Access Point)) of the network closer to the UE
- Edge Computing Characteristics
 - Very short delay time with high QoS support
 - Low usage of the network resources
 - Low battery usage of IoT & Smart Devices
 - More Complex: Prediction and pre-fetching of contents and control functions needed

Drawbacks of MCC

- Conventional MCC
- Cloud services to the mobile device are accessed via the Internet connection
- Conventional MCC Characteristics
 - Long delay time & QoS performance is low
 - High usage of the network resources
 - High battery usage of IoT & Smart Devices
 - Significant packet interarrival time jitter
 - Large DBV (Delay Bound Violation) for real-time multimedia services
 - Multimedia :
 - » Distance time between the frames and packets received may not be met
 - » one frame → 30 packets → per second one frame: dividing the one second for 30 segments then reconstruct, then at exact frame of moving video/audio, delays leads to delay bound violation, next frame will be received while reconstructing the previous frame.

IoT Mobile Edge computing technology

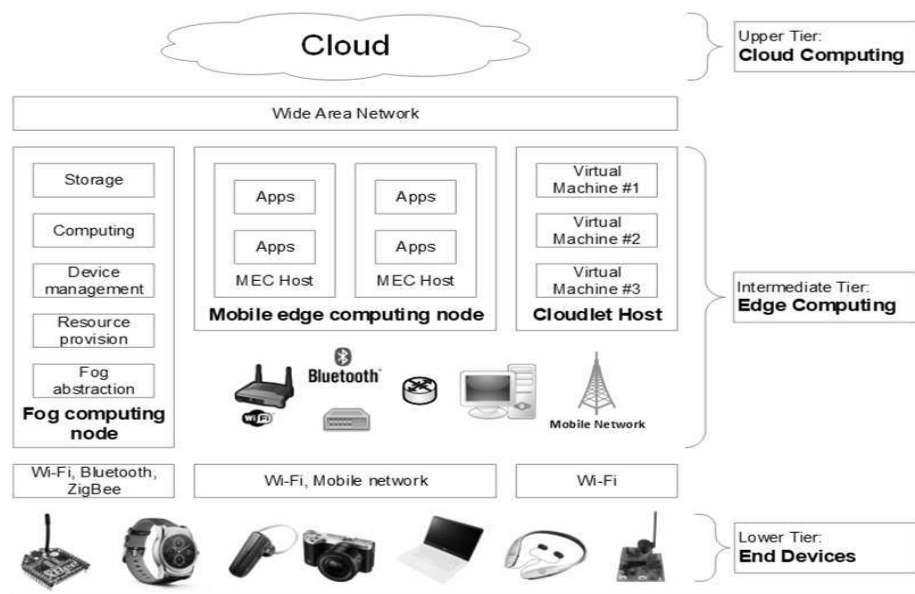


Comparison of MCC & Edge computing

Technical aspect	MCC	Edge computing
Deployment	Centralized	Distributed
Distance to the UE	Far	Close
Latency	Long	Short
Jitter	High	Low
Computational Power	Abundant	Limited: uses small devices
Storage Capacity	Abundant	Limited

EDGE COMPUTING TECHNOLOGY

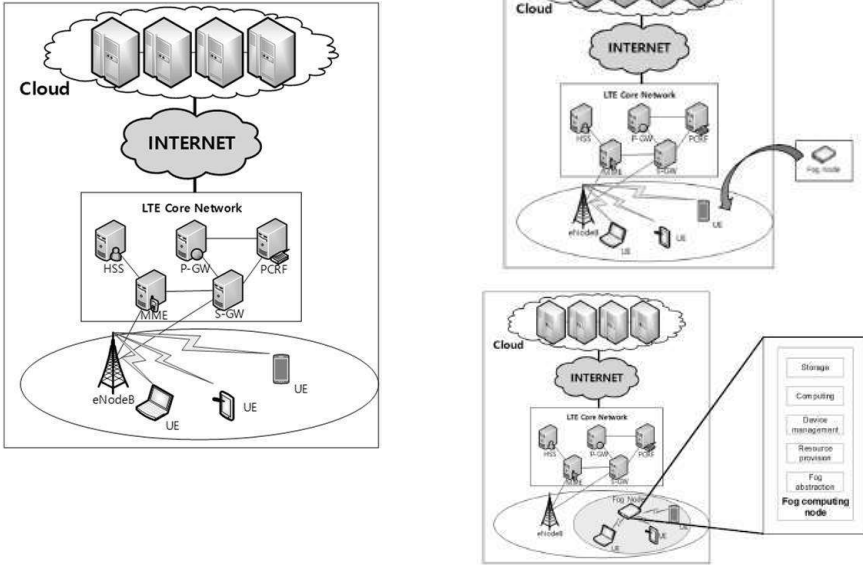
IoT Edge computing technology



Type of cloud technologies in edge computing

- Fog Computing
- Mobile Edge computing (MEC)
- Cloudlet

Fog Computing



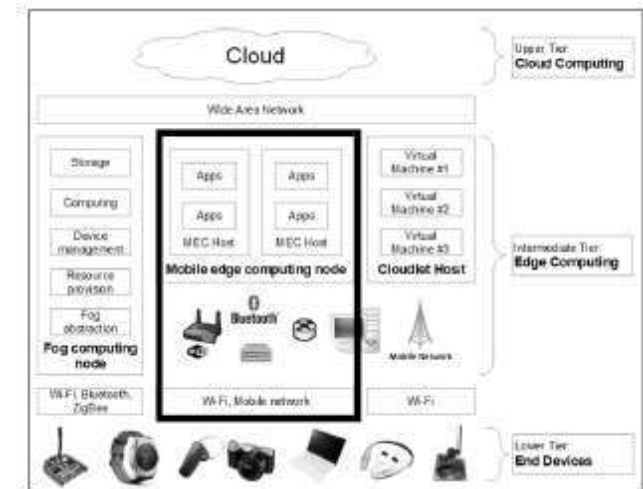
Characteristics

- **Decentralized Computing infrastructure** is based on FCNs (Fog Computing Nodes)
- FCNs **can be placed anywhere between the end devices and cloud**
- FCNs **are heterogeneous (not one form, one protocol and one structure) in nature**, and can include various functional elements (some as storing, memory, services)
 - **Routers, switches, AP (Access Point), IoT gateways, set-top box, etc.**
 - We can have the above components can be converted as Fog device.
- FCN heterogeneity enables support for different protocols as well as IP and non-IP based access to Cloud services
- Uniform **Fog abstraction layer** has functions to support **resource allocation, monitoring, security, device management, storage**, and compute services for various types of end devices

Definition

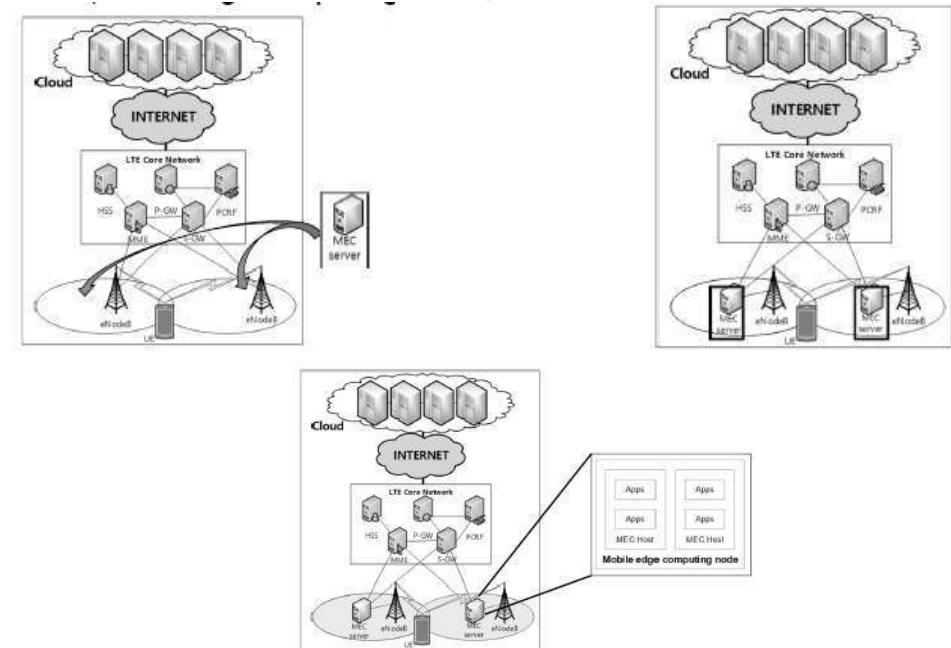
- Fog Computing or Fog Networking
- Uses one or more collaborative end-user
 - Clients or near-user Edge devices
- Fog support services
 - Substantial amount of storage
 - Instead of storing in a cloud data center (stores locally)
 - Communications
 - Instead of routing over the Internet backbone
 - Control, configuration, measurement and management through Fog Device
 - Instead of being controlled by Internet gateways or LTE S-GWs or the P-GW

Mobile Edge Computing(MEC)



- The MEC is based on **mobile based networks**
- It can use wifi based networks on MEC node as **MEC Host that had application inside**

MEC



Definition

- MECs enable cloud computing and IT services at the edge of the mobile cellular network
- Why use MECs?
 - Faster cloud services to mobile UEs
 - Reduces network congestion
 - MCC every user request need to go to internet but that is not be done here
 - Number of users connected is directly proportional to the traffic.
 - Stores the information locally
 - Delivering the content by storing the data that would be used by the clients later
 - Delivering the content to the user quickly
 - If it finds the information available share it among the users
 - More reliable application support
 - If a need a transmit a packet, as MEC is near to UE device, it will deliver very effectively and reliable data transfer support
 - No MEC, need to cloud to retransmit the packets

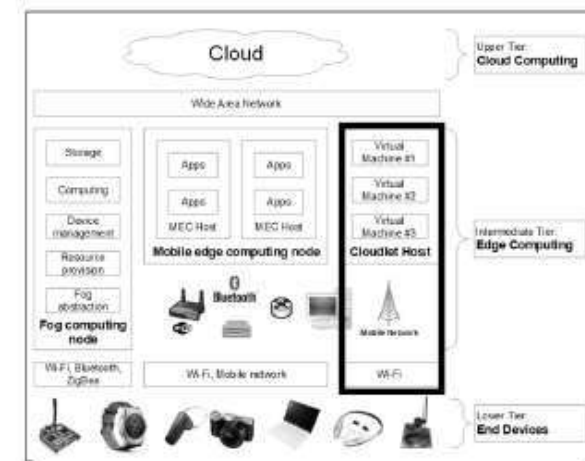
Characteristics

- MECs enable cellular operators to open their RAN (Radio Access Network) to authorized third-parties (e.g., application developers, content providers, etc.)
- MEC servers offer real-time information
 - Network information (e.g., network load and capacity)
 - Information on the end devices connected to the servers (e.g., location information, etc.)
 - Assisted GPS techniques and location base services Very effective.
- MEC nodes (or servers) are usually co-located with a RNC (Radio Network Controller) or macro BS (Base Station)
 - eNB BS in LTE and LTE-A 4G networks
 - gNB BS in 5G networks
- Servers may run multiple MEC Hosts
 - Some hosts will be divided into various services.
- MEC Hosts can perform computation and storage through virtualized interfaces

MEC Controller

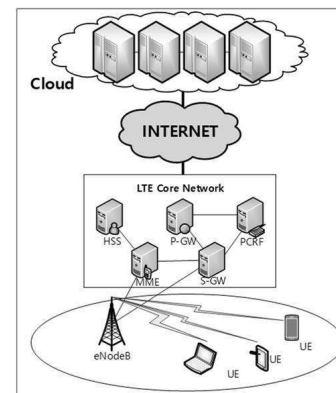
- MEO (Mobile Edge Orchestrator)
 - Manages MEC hosts
 - Controls information flow for services offered by each MEC host
 - Controls resources and network topology
 - Manages the Mobile Edge applications

Cloudlet

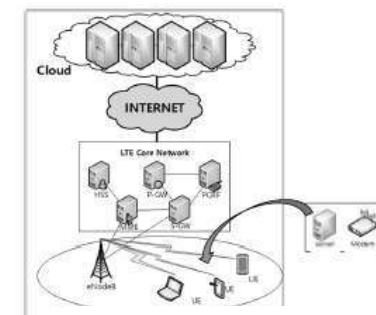


Cloudlet

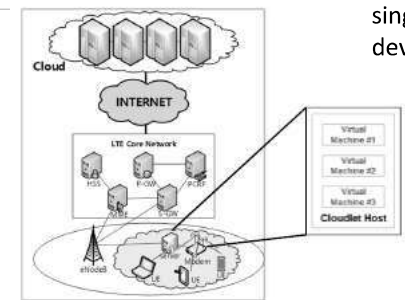
- Access through **wifi networks** and inside with virtual machines
- Mainly with cloud technology



Connecting as cloudlet Host device with all other local devices



A cloud server interconnected with modem device May be as single device



Definition

- Mobility-enhanced small-scale cloud data centre
- Located at the edge of the Internet nearby the mobile devices
 - In some cases Some mobile devices could be acting as cloudlets
- Supporting resource-intensive and interactive mobile applications
- Provides powerful computing resources to mobile devices with low latency
- Used between UE → Cloudlet → Cloud
 - Middle of the mobile based cloud computing architecture
- Cloudlet is a data center in a box that brings the cloud closer to the UE
 - Really like cloud but as limited features and is miniature cloud device providing Wi-Fi technology features etc.
- Cloudlets use a VM (Virtual Machine) to provision resources for UEs in real-time over Wi-Fi networks
 - Direct one-hop Cloud access → Low latency

Cloudlet Architecture: 3 Layers

- Cloudlet layer
 - Group of co-located nodes
 - Managed by a Cloudlet Agent
- Node layer
 - Multiple Execution Environment(s) running on top of the OS (Operating System)
 - Managed by a Node Agent
- Component layer
 - Includes a set of services that interface to the (higher layer) execution environment

comparison

	Fog Computing	Mobile Edge Comp.	Cloudlet Computing
Node Device	Routers, Switches, Access points, Gateways	Servers running in base stations	Data center in a box
Node location	Varying between end devices & cloud	Radio network controller/ Macro base station	Local/outdoor installation
Software Architecture	Fog abstraction layer	Mobile orchestrator	Cloudlet agent
Context Awareness	Medium	High	Low
Proximity	One or multiple hops	One hop	One hop
Access Mechanisms	Bluetooth, Wi-Fi, Mobile networks	Mobile networks	Wi-Fi
Internode Comm.	Supported	Partial	Partial

Physical proximity	High	Fog computing, Cloudlet computing
	Low	Mobile edge computing
Power consumed	High	Mobile edge computing
	Low	Fog computing, Cloudlet computing
Computation time	High	Fog computing
	Low	Mobile edge computing, Cloudlet computing
Context awareness	High	Mobile edge computing
	Low	Fog computing, Cloudlet computing
Logical proximity	Ensured	Cloudlet computing
	Maybe	Fog computing, Mobile edge computing
Non-IP support	Yes	Fog computing
	No	Mobile edge computing, Cloudlet computing

Scenario

- Design the smart logistics for any application based on cloud based, edge based techniques.
 - Write about cloud based technique based on scenario and its diagram
 - Write about edge based technique based on scenario and its diagram

Reference

- IoT Wireless & Cloud Emerging Technology
(course era) by Prof. Jong-Moon Chung